

An Empirical Evaluation of Zero-Shot GPT-4o-mini for Java Unit-Test Generation

Le The Khang, Huynh Cao Phuoc, Pham An Khang, Nguyen Thi Nhu Y, and Do Long Vy
Software Testing and Research-Based Learning (SE1944)
FPT University, Vietnam

Abstract—Large language models can generate unit tests directly from source code, but a successful API response does not by itself establish that the resulting suite compiles, passes, or detects faults. This study evaluates zero-shot GPT-4o-mini unit-test generation for 63 Java functions from a local HumanEval-Java transformation. We measure executable-suite rate, branch coverage, mutation score, and paired differences against retained EvoSuite test suites measured under 1-, 3-, and 5-minute budgets. The generation service returned content for all 63 targets, and one repair pass was attempted where needed; however, only 14 generated suites executed successfully. Across all targets, GPT reached 18.90% branch coverage and 16.21% mutation score. The dual threshold was reached by 13/63 functions. On the executable paired subset, no GPT–EvoSuite difference remained statistically significant after Holm correction. The result highlights execution validity as the main constraint in this zero-shot setting and separates aggregate capability from pass-conditioned paired comparisons.

Index Terms—large language models, unit test generation, mutation testing, branch coverage, EvoSuite, empirical software engineering

I. INTRODUCTION

Unit tests are a practical safeguard against regressions, but writing them manually requires developers to identify representative inputs, boundaries, and expected outcomes. Code-oriented language models make direct test generation possible: a model can receive a function and produce a JUnit class in one call. The original HumanEval work established a widely used benchmark for code-generation evaluation [1]; however, test generation has an additional requirement. A generated test must be executable and its assertions must agree with the system under test (SUT). Text returned by an API is therefore only the first stage of the evaluation.

This study evaluates whether a fixed zero-shot prompt to GPT-4o-mini can generate useful JUnit 4 suites for Java functions. We use 63 functions from the repository’s HumanEval-Java transformation. The study reports branch coverage and mutation score, rather than relying on a single coverage number. Branch coverage records whether decision outcomes were executed; mutation score records the proportion of injected changes killed by the suite. Together, the metrics distinguish broad execution from fault-detection strength.

We compare GPT-generated suites with retained EvoSuite suites. EvoSuite is a search-based Java test-generation tool [2]; its existing 1-, 3-, and 5-minute archives were measured again using the same Maven, JaCoCo, and PIT pipeline. This is an operational technical comparison, not a student benchmark.

The originally planned student-written comparator has no validated per-function measurements in the same pipeline and is therefore explicitly deferred.

The central research question is whether zero-shot GPT-4o-mini tests meet predefined coverage and mutation thresholds and how their metrics differ from EvoSuite on the same SUTs. We ask five subquestions: (RQ1) branch-coverage threshold attainment; (RQ2) mutation-score floor and target attainment; (RQ3) paired GPT–EvoSuite differences for each budget; (RQ4) simultaneous threshold attainment; and (RQ5) the technical failure pattern that blocks valid execution.

This paper contributes: (1) a traceable execution report for 63 Java targets, including raw API usage, repair records, and class-level metrics; (2) a paired, budget-specific comparison with existing EvoSuite suites; and (3) an explicit analysis boundary that prevents pass-conditioned results from being generalized to all generated suites. The repository retains the prompts, scripts, raw result files, and analysis notebook needed to reproduce the reported analysis.

II. BACKGROUND AND RELATED WORK

Automated test generation has long used search-based techniques. EvoSuite generates Java unit tests and assertions while optimizing an objective such as coverage [2]. It is consequently an appropriate technical reference point for a Java experiment, but its performance should not be interpreted as a measurement of human-written tests.

Language-model evaluation introduced code-generation benchmarks such as HumanEval [1]. The local HumanEval-Java material used here transforms that setting into Java programs with correct and buggy variants. In contrast to program synthesis, test generation must construct an executable oracle. A plausible-looking assertion can make the test suite fail even when the SUT is correct; this motivates reporting compile/pass status separately from coverage metrics.

The study uses branch coverage from JaCoCo and mutation score from PIT. These measurements answer different questions. A suite can execute decision paths but still fail to distinguish a mutated implementation from the original. Mutation score is therefore reported alongside branch coverage rather than inferred from it. This distinction is also central to mutation-guided generation: Wang et al. report that a suite can reach 100% coverage while killing only 4% of mutants, motivating validation beyond structural reachability [3].

Our inferential design follows common guidance for statistical comparisons in software engineering [4]. The RQ3 comparisons are paired by SUT, two-sided, and evaluated independently for each EvoSuite budget and metric. The six RQ3 p-values are adjusted with the Holm procedure [5]. This design retains the pairing rather than comparing unrelated aggregate averages.

The thresholds in this study are intentionally not presented as universal targets. Huang et al. report mean branch coverage of 30.22% and mutation score of 40.21% across 12 LLMs on the 3,909-function ULT benchmark [6]. We use those values as externally derived reference targets, while acknowledging the differences in language and benchmark. The 4.00% mutation floor is a deliberately low operational screen: it prevents a suite with essentially no observed fault detection from being counted as dual success, but it is not a claim about adequate industrial test quality.

A. Why Execution Status Must Be Reported

An executable test suite is a prerequisite for interpreting a test-quality metric. A test class may fail before it reaches the SUT, may contain an incorrect expected value, or may use an unsupported JUnit API. Treating these outcomes as merely formatting defects would overstate an LLM’s practical usefulness. We therefore keep three layers of evidence: generation status records whether the service returned content; execution status records whether Maven compiled and passed the suite; and quality metrics report coverage and mutation outcomes after this gate.

This separation is especially important for paired comparisons. If only successful GPT suites are paired, the paired analysis estimates a conditional question: how do the suites that survived validation compare with EvoSuite? It does not estimate the expected quality of an arbitrary GPT response. Conversely, assigning a score only to the passing subset would hide the operational cost of invalid suites. The full-corpus and pass-conditioned views are thus reported side by side throughout this paper.

B. Comparator Scope

EvoSuite is a legitimate technical baseline because it generates Java suites automatically under a specified search budget [2]. The measured 1-, 3-, and 5-minute archives make that budget explicit. However, an automatic search-based suite and a student-written suite answer different empirical questions. The present comparator is intentionally narrow: it evaluates the current GPT protocol against EvoSuite on identical SUTs. The study does not infer human capability from either tool.

III. METHODOLOGY

A. Subjects and Protocol

The population consists of 63 representative Java functions from the repository’s HumanEval-Java transformation. The correct implementation of each function is the SUT. For each target, the generation script supplied its complete source

code to `gpt-4o-mini-2024-07-18` with a fixed zero-shot instruction. The prompt required a single executable JUnit 4 class in package `humaneval.correct`, correct assertions, and coverage of normal, boundary, and error cases. Temperature was fixed to zero. No output-token cap was set in the API call.

The run retained a timestamped API log and a CSV record for each target. The initial call was retried only for retryable service failures. One subsequent repair invocation was permitted for syntactic or assertion issues. Generated tests were then compiled and executed with Maven. Tests that remained invalid were archived rather than deleted, and their execution status was retained in the result set.

The comparator consists of existing EvoSuite archives generated under 1-, 3-, and 5-minute budgets. No EvoSuite generation was performed for this study. Each archive contains one suite per SUT and was measured against the same target set with the same Maven, JaCoCo, and PIT configuration. Temurin JDK 8 was used for this measurement because EvoSuite 1.0.6 requires `tools.jar`.

B. Evidence Retention and Postprocessing

The pipeline records a row per SUT for generation, repair, execution status, and class-level metrics. The primary files are the full API-output CSV, repair-output CSV, compile-status CSV, GPT class-metrics CSV, three EvoSuite class-metrics CSVs, a measurement manifest, and the derived summary CSV. The notebook reads these retained records rather than manually copying values into a report. A validation script checks the number of distinct SUTs, status partition, coverage numerators, mutation numerators, comparator pass counts, RQ3 rows, notebook outputs, and figure resolution.

Postprocessing does not repair the scientific record silently. Generated test classes are moved into timestamped archive or invalid-test locations after measurement rather than deleted. The active test directory is cleared only after preservation. This matters because a later reader must be able to distinguish a pipeline configuration change from a changed test suite. The paper reports the initial API run and the permitted repair pass separately, so the final execution gate is not confused with a single unaided model response.

C. Measures and Decision Rules

For every SUT, JaCoCo produced branch-covered and branch-total counts. Branch coverage is the covered-to-total ratio. PIT produced killed-mutant and eligible-mutant counts; mutation score is their ratio. A suite was executable only if its test class compiled and passed. The report retains zero or unavailable class-level outcomes for non-executable suites as defined by the pipeline, while the paired RQ3 analysis includes only SUTs that passed in both tools.

RQ1 tests whether the per-SUT GPT branch score exceeds 30.22%. RQ2 tests the mutation score against an empirical 4.00% floor and a 40.21% target. RQ4 tests whether more than half of the 63 SUTs simultaneously pass, reach branch coverage of at least 30.22%, and reach mutation score of at

least 4.00%. RQ1 and RQ2 use one-sided Wilcoxon signed-rank tests; RQ4 uses an exact binomial test. The significance level is $\alpha = 0.05$.

For RQ3, GPT and EvoSuite scores are paired by SUT for branch coverage and mutation score at each budget. The test is a two-sided paired Wilcoxon signed-rank test. Three technical mutation exclusions are omitted from mutation-score tests. The six p-values are Holm-adjusted [5]. We report the raw and adjusted p-values, the paired and non-zero-ranked sample sizes, percentage-point difference, and rank-biserial effect size. A comparison is considered statistically significant only when the adjusted p-value is below 0.05.

The report preserves two denominators for RQ3. The paired denominator counts SUTs that executed in both tools. The ranked denominator counts non-zero differences used by the signed-rank statistic. When many pairs have identical scores, the latter can be much smaller. Reporting both prevents a small ranked sample from being mistaken for 63 independent observations.

D. Generation Prompt (Verbatim Template)

The following is the exact template passed as the user message. The two placeholders were instantiated separately for each SUT; `{source_code}` was replaced by the complete source of that target. No example test, previous model response, or few-shot demonstration was included.

You are an expert Java developer and software tester. Your task is to write a comprehensive JUnit 4 test suite for the following Java class. Strictly adhere to the following requirements:

1. Generate test cases using JUnit 4 (use `org.junit.Test`, `org.junit.Assert`).
2. Do not use JUnit 5 or other test frameworks.
3. Test all logical paths, edge cases, boundary values, and potential error conditions.
4. Ensure all assertions are correct and correspond exactly to the expected behavior of the correct code.
5. The test class must be named `{class_name}_GPTTest`, corresponding to the target class `{class_name}`.
6. The test class must be in package `humaneval.correct`.
7. You should import any required packages (like `java.util.*`).
8. Provide only the executable Java test class code. Do not include any markdown explanations, text wrapping, or extra commentary.

Source Code: `{source_code}`

E. Reproducibility and Protocol Amendment

The analysis notebook, derived CSVs, figures, API usage records, execution statuses, archive manifest, and validation report are stored in the replication package. The original proposal motivated a future comparison with student-written tests. Comparable per-SUT student measurements were unavailable, so that comparison is deferred. The present paper consequently makes no numerical claim about student performance and never treats EvoSuite as a student proxy.

IV. RESULTS

A. Execution Validity (RQ5)

The initial API generation returned content for all 63 targets, with an initial recorded cost of \$0.030982. A single repair pass recorded 55 repair calls costing \$0.036773. After compilation and execution, 14 of the 63 GPT suites passed.



Fig. 1. Execution status of all 63 GPT-generated suites.

Forty-seven suites failed assertions and two failed compilation. Thus, assertion failures accounted for 47 of the 49 invalid suites (95.9%), making execution-valid oracle construction the predominant observed limitation.

B. Threshold Results (RQ1, RQ2, and RQ4)

Across all 63 targets, GPT reached 144/762 covered branches (18.90%) and killed 135/833 eligible mutants (16.21%). Table I reports the predefined hypothesis tests with the effect size and analysis denominator for each RQ. RQ1 was not supported. The aggregate mutation score exceeded the 4.00% floor, but the per-SUT one-sided test did not support RQ2 at either the floor or target. For RQ4, only 13/63 SUTs simultaneously passed and met both thresholds (20.63%); the exact binomial result did not support a majority.

For RQ1 and RQ2, r_{rb} is the rank-biserial effect of the per-SUT difference from the stated threshold; negative values indicate that the ranked differences favor values below the threshold. For RQ4, RD is the observed dual-success rate minus the hypothesized 50% majority rate. RQ5 is descriptive rather than an inferential hypothesis: its denominator is all 63 targets, and it reports 14 passing suites, 47 assertion failures, and two compilation failures.

C. Paired Comparison with EvoSuite (RQ3)

All retained EvoSuite suites passed for each budget. Its aggregate branch coverage rose from 90.29% at one minute to 99.34% at five minutes; mutation score rose from 73.23% to 82.11%. The paired analysis is deliberately narrower: it is conditioned on the 14 GPT suites that also passed. This selection is why the pass-conditioned GPT means in Table II should not be read as full-set GPT performance.

No RQ3 comparison was significant after Holm correction. The smallest adjusted p-value was 0.375053 for the five-minute mutation comparison. Figure 3 visualizes the paired measurements. For branch coverage, only two non-zero paired differences were ranked; this limited variation is reported directly rather than interpreted as evidence of equality.

The full EvoSuite aggregates provide a separate descriptive baseline. At one minute, EvoSuite covered 688/762 branches

TABLE I
THRESHOLD AND DUAL-SUCCESS RESULTS.

RQ	Test	N	Aggregate/Rate	p	Effect	Decision
RQ1	Branch > 30.22%	63	18.90%	0.942967	$r_{rb} = -0.215$	Not supported
RQ2 floor	Mutation > 4.00%	60	16.21%	0.952447	$r_{rb} = -0.233$	Not supported
RQ2 target	Mutation > 40.21%	60	16.21%	0.997231	$r_{rb} = -0.387$	Not supported
RQ4	Dual success > 50%	63	13/63 (20.63%)	0.9999996	$RD = -29.37$ pp	Not supported

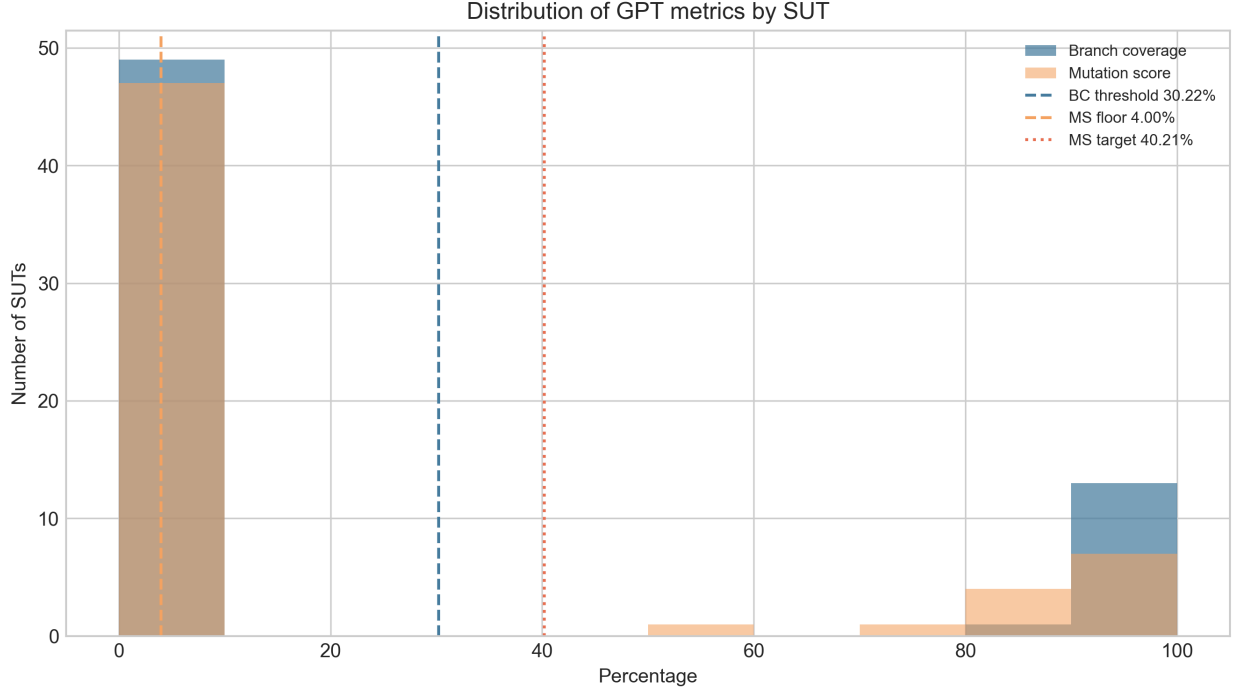


Fig. 2. Distribution of GPT branch coverage and mutation score across the full SUT set.

TABLE II
PAIRED GPT-EvoSUITE RESULTS. N_p/N_r DENOTES PAIRED/NON-ZERO-RANKED SUTs; Δ IS GPT MINUS EvoSUITE IN PERCENTAGE POINTS.

Budget	Metric	N_p/N_r	GPT mean	Evo mean	Δ	Raw p	Holm p	r_{rb}
1 min	Branch	14/2	98.09%	99.40%	-1.32	0.179712	0.898562	-1.000
1 min	Mutation	13/7	88.21%	79.14%	+9.07	0.204084	0.898562	+0.536
3 min	Branch	14/2	98.09%	99.40%	-1.32	0.179712	0.898562	-1.000
3 min	Mutation	13/7	88.21%	78.20%	+10.01	0.236724	0.898562	+0.500
5 min	Branch	14/2	98.09%	99.40%	-1.32	0.179712	0.898562	-1.000
5 min	Mutation	13/7	88.21%	82.40%	+5.81	0.062509	0.375053	+0.786

(90.29%) and killed 610/833 mutants (73.23%). At three minutes, the corresponding values were 730/762 (95.80%) and 638/833 (76.59%). At five minutes, they were 757/762 (99.34%) and 684/833 (82.11%). All 63 archived suites passed at every listed budget. These totals demonstrate the technical consistency of the retained comparator; they are not included in the paired hypothesis-test denominator unless the GPT counterpart also passed.

The observed difference between aggregate and pass-conditioned GPT scores is material. The full corpus includes every generated suite, including the 49 that did not pass. The RQ3 mean branch score of 98.09% and mutation score of

88.21% therefore characterize a selected subset, not the overall generator. This is why Tables I and II answer complementary questions rather than conflicting ones.

V. DISCUSSION

The primary finding is the gap between generation availability and executable test quality. GPT returned content for every target, yet 49 suites remained invalid after the allowed repair step. This makes the full-set branch coverage and mutation score modest, and it prevents the predefined RQ1, RQ2, and RQ4 hypotheses from being supported. The result is useful even though it is negative: in this protocol, zero-shot

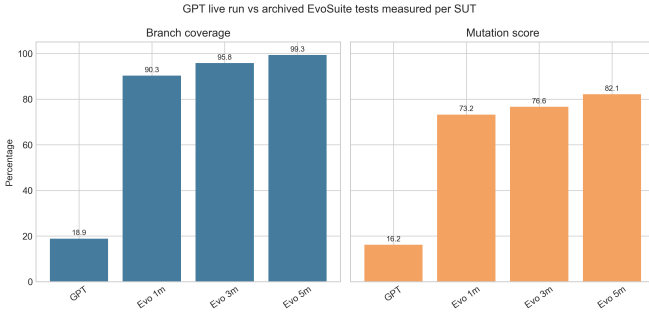


Fig. 3. Pass-conditioned paired GPT-EvoSuite comparison.

generation did not consistently produce valid test oracles for the chosen Java targets.

The paired results require a different interpretation. When both GPT and EvoSuite suites pass, the selected GPT suites have high coverage and mutation scores, and the five-minute mutation comparison has the smallest raw p-value. However, the paired sample contains only 14 executable GPT suites (13 for mutation) and the branch analysis has two non-zero ranked differences. After controlling the six RQ3 comparisons with Holm adjustment, no difference is significant. These results do not establish parity between tools; they establish that this run lacks statistically significant evidence of a difference on the small pass-conditioned subset.

For practice, the observed workflow suggests that an LLM can be a useful candidate-test generator only when followed by strict compilation, execution, and oracle validation. A raw response should not be counted as a successful test suite. Search-based generation remained consistently executable on the archived EvoSuite suites under the measured budgets, while the LLM pipeline’s main improvement opportunity is repair or feedback targeted at assertion validity.

The study also clarifies the scope of the comparator. EvoSuite supplies a reproducible engineering baseline on the same SUTs; it does not measure what students write. A future study should collect student-written suites and evaluate them with the identical build, coverage, and mutation pipeline before making any GPT-student claim.

A. Practical Implications

For a developer or teaching team, the immediate implication is not to reject LLM assistance altogether. Instead, the generation step should be treated as a proposal mechanism. A safe workflow keeps the generated source, compiles it in isolation, executes it against the known-correct SUT, and measures it only after it passes. A failure category should remain visible in the final dashboard. This workflow prevents a high-level language-model success rate from obscuring the cost of manual debugging or incorrectly asserted behavior.

The result also motivates feedback-guided repair. The present run allowed only one repair invocation and did not use coverage or mutation feedback to steer subsequent generation. Because assertion failures dominate the observed invalid

outcomes, a next protocol can give the model the compiler output, failing assertion, or minimized counterexample and then evaluate whether a bounded repair policy raises the executable-suite rate without inflating API cost. Such a follow-up should predefine the maximum repair attempts and report the separate contribution of each attempt.

B. Interpretation of Negative Findings

The negative threshold results should not be described as evidence that LLMs cannot generate useful tests. They establish a narrower fact: this model snapshot, fixed prompt, and zero-shot repair policy did not meet the preregistered thresholds on this 63-SUT setting. The evidence is informative because it identifies which condition failed most often. A future comparison can alter one factor at a time—for example, structured few-shot prompting, test-feedback loops, or domain-specific context—while retaining the same execution and metric gates.

C. Measurement and Reporting Implications

This evaluation also demonstrates why a single aggregate number is insufficient for an LLM testing study. The all-target scores answer a deployment-oriented question: if a team runs the configured process once for every selected function, what quality is obtained without discarding failures? The pass-conditioned paired analysis answers a different diagnostic question: when both approaches produce runnable suites for the same target, is there evidence that their measured adequacy differs? Mixing the denominators would hide the practical failure rate in the former question or unfairly attribute failures to a tool in the latter question. The report therefore keeps the 63-target population, the executable GPT subset, the paired subset, and the non-zero ranked subset explicit in both prose and tables.

The use of mutation analysis has a similar implication. Branch coverage indicates whether tests traverse decisions, whereas mutation score asks whether their assertions expose seeded behavioral changes. A generated suite can execute many branches while still accepting incorrect values, especially when assertions are weak, over-general, or based on an incorrect interpretation of the specification. Conversely, a small test suite may kill a high proportion of the mutants reachable in a simple function. Reporting the two measures together does not make them interchangeable; it makes their distinct limits visible. For this reason, future extensions should retain the class-level counts behind each percentage and should classify the major invalid-suite modes, rather than only reporting a mean coverage score.

Finally, the study treats reproducibility as part of the result rather than a later packaging task. The retained generation records make it possible to rerun the computations, inspect a particular invalid class, and distinguish a model-output problem from a build or instrumentation problem. This is particularly important for API-based experiments because model aliases, service behavior, and local dependencies can change over time. A replication should preserve the dated

model identifier, prompt template, retry and repair policy, target manifest, compiler version, metric-tool versions, and exact decision rules before comparing a new run with this baseline.

D. Operational Cost and Selection Effects

The recorded service cost is small for this run, but cost should not be interpreted independently of validity. The initial calls cost \$0.030982 and the permitted repair calls cost \$0.036773. These figures describe only the API portion of the workflow. They omit the local time needed to compile, diagnose a failure, measure the suite, and decide whether further human intervention is acceptable. In a practical setting, a comparison between test-generation methods should therefore report at least three dimensions: generation expenditure, executable-suite yield, and post-generation engineering effort. A cheap response that fails its assertions may still be expensive when it creates review work for a developer.

The RQ3 subset creates a second, statistical form of selection. By definition, it excludes the 49 GPT suites that did not execute. This is appropriate for a paired measurement of score differences because coverage and mutation scores cannot be compared fairly when one side has no valid test class. It is not appropriate as a stand-alone claim about end-to-end test generation. The full-set and paired views should consequently be read together. The former bounds the outcome a user receives from the configured pipeline; the latter describes the quality of the successful cases. Presenting both avoids the common but misleading practice of reporting high coverage only after silently filtering failed generations.

E. Design for a Follow-Up Study

A confirmatory follow-up should change as little as possible while addressing the major observed bottleneck. One design would hold the 63-SUT manifest and the JaCoCo/PIT measurement configuration constant, then compare fixed zero-shot, fixed few-shot, and bounded feedback-guided prompts. Each condition should use the same dated model identifier, maximum number of repair attempts, temperature, and token-accounting policy. The primary outcome should remain the executable-suite rate; branch and mutation scores should be reported for both the full corpus and the passing subset. This design would identify whether additional context or feedback improves validity rather than merely selecting a different benchmark or metric denominator.

The deferred student comparison also needs a separate protocol instead of an inferred proxy. Student-written test suites should be collected with consent, mapped to the same functions, built in the same environment, and measured without modifying their assertions. The study should preregister how incomplete or non-compiling submissions are counted, how multiple student attempts are handled, and whether the comparison unit is a student, a suite, or a function. Only after those choices are fixed can an LLM-student statement be statistically and ethically interpretable. Until then, EvoSuite remains useful as a reproducible search-based baseline, not evidence about students.

VI. THREATS TO VALIDITY

Internal validity. The protocol fixed the model snapshot, prompt, temperature, target sources, and measurement tooling. Nevertheless, model-service behavior and repair outcomes can vary over time. The retained API usage records, raw responses, and timestamps mitigate but do not remove this risk.

Construct validity. Branch coverage and mutation score are proxies for test quality, not complete definitions of it. We mitigate single-metric bias by reporting both measures and execution status. Non-executable suites are not treated as valid evidence of coverage quality.

Conclusion validity. RQ3 is conditional on successful execution, resulting in 14 branch pairs and 13 mutation pairs. The number of non-zero branch differences is particularly small. We report raw and Holm-adjusted p-values, effect sizes, and both paired and ranked sample sizes, but the study has limited power for this subset.

External validity. The 63 Java functions come from one transformed benchmark and one fixed zero-shot model configuration. The findings should not be generalized to other models, prompting strategies, repositories, languages, or student populations without further measurements. In particular, the absence of validated student per-function data means no conclusion about student-written tests is supported.

Measurement validity. The aggregate score denominators are produced by the configured JaCoCo and PIT runs. Three technical mutation exclusions are removed only from the appropriate Wilcoxon samples and are stated in the summary CSV. The project retains exact numerators and class-level rows so that a future change in tool version or inclusion rule can be detected. The study does not claim that these two metrics capture readability, maintainability, or developer effort.

Protocol deviation. The original motivation included a student-written benchmark. It is not substituted with EvoSuite in this paper. The operational amendment limits the technical comparison to retained EvoSuite archives and records instructor confirmation as pending. This disclosure reduces the risk that an implementation convenience is later reported as a completed human-subject comparison.

VII. CONCLUSION

We evaluated zero-shot GPT-4o-mini test generation on 63 HumanEval-Java functions using execution status, branch coverage, mutation score, and paired EvoSuite comparisons. GPT produced an API response for all targets, but only 14 suites were executable after one repair pass. The full-set branch coverage was 18.90%, mutation score was 16.21%, and 13/63 functions achieved the dual threshold; RQ1, RQ2, and RQ4 were not supported. For RQ3, no GPT-EvoSuite comparison was significant after Holm correction on the pass-conditioned subset. Assertion failures were the predominant observed obstacle.

Future work should evaluate feedback-guided or multi-turn repair prompts, repeat the protocol across additional Java benchmarks, and collect verified student-written suites

under the same pipeline. These steps would test whether the execution-validity bottleneck can be reduced and would enable the deferred student comparison without conflating human and search-based baselines.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [2] G. Fraser and A. Arcuri, “EvoSuite: Automatic test suite generation for object-oriented software,” in *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, 2011. [Online]. Available: <https://doi.org/10.1145/2025113.2025179>
- [3] G. Wang, Q. Xu, L. Briand, and K. Liu, “Mutation-guided unit test generation with a large language model,” *IEEE Transactions on Software Engineering*, vol. 52, no. 5, pp. 1657–1671, 2026. [Online]. Available: <https://doi.org/10.1109/TSE.2026.3682975>
- [4] A. Arcuri and L. Briand, “A practical guide for using statistical tests to assess randomized algorithms in software engineering,” in *Proceedings of the 33rd International Conference on Software Engineering*, 2011. [Online]. Available: <https://doi.org/10.1145/1985793.1985795>
- [5] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: <https://www.jstor.org/stable/4615733>
- [6] D. Huang, J. M. Zhang, M. Harman, Q. Zhang, M. Du, and S.-K. Ng, “Benchmarking LLMs for unit test generation from real-world functions,” *ACM Transactions on Software Engineering and Methodology*, 2026. [Online]. Available: <https://doi.org/10.1145/3805043>